OpenCV·YOLO 기초와 RailSafeNet 검토

Oct 6, 2026 · @빈

결론: YOLO는 "무엇이 어디 있나"를 찾고, OpenCV는 영상을 읽고 선로를 그리고 거리를 계산하는 손발입니다. RailSafeNet은 MIT 라이선스라 참고·인용해도 되지만, 거기 딸린 YOLO(AGPL-3.0)와 SegFormer 가중치(비상업 전용) 조건 때문에 우리 코드 저장소를 공개하고 출처를 밝히는 쪽이 가장 안전합니다. 팀 로드맵(OAK-D Lite + 라즈베리파이 5)을 기준으로 정리했습니다.

1. OpenCV와 YOLO는 어떻게 나눠 쓰나

한 줄로 나누면 YOLO는 눈(무엇이 어디 있나), OpenCV는 손(영상 다루기·선 긋기·픽셀 계산)입니다. 둘은 경쟁이 아니라 한 프로그램 안에서 같이 씁니다.

도구	정체	우리 프로젝트에서 맡는 일	처음 해 볼 것
YOLO (Ultralytics)	학습된 딥러닝 모델. 사진을 넣으면 사람·차·자전거 같은 물체의 박스와 종류를 돌려줌	사람·자전거·차량 검출, `track`으로 같은 물체에 ID 유지, `seg` 모델로 레일 영역 분할	녹화한 트램 영상에 `model.track()` 돌려 보기
OpenCV	영상 처리 라이브러리. 딥러닝이 아니라 "그림 다루는 도구 상자"	카메라·영상 파일 읽기, 박스·위험 통로 그리기, 화면 출력, 레일 엣지(Canny·HoughLinesP) 검출, 픽셀 → 미터 계산	`cv2.VideoCapture`로 영상 읽고 `cv2.line`으로 선 긋기
카메라 내장 칩 (OAK-D Lite)	카메라 안에 있는 작은 AI 칩 + 스테레오(두 눈) 깊이 센서	YOLO를 카메라 칩에서 직접 돌리고, 각 박스까지의 3D 거리(X·Y·Z)를 같이 줌	DepthAI 공식 예제 "spatial detection" 실행

한 프레임은 이렇게 흐릅니다: OpenCV(또는 OAK-D)가 영상을 받는다 → YOLO가 물체를 찾는다 → OpenCV와 우리 코드가 선로와 거리를 계산한다 → 위험 단계를 정한다 → OpenCV로 화면에 그린다.

주의할 점은 OAK-D Lite의 깊이 측정이 가까운 거리(로드맵 기준 약 0.4~8 m)에서만 정확하다는 것입니다. 그보다 먼 곳은 레일 폭(궤간 1.435 m)을 자로 써서 카메라 한 대로 거리를 추정해야 하고, 이 부분은 RailSafeNet 방식이 도움이 됩니다(3장).

2. 공개 코드를 가져다 써도 되나

쓸 수 있습니다. 다만 "상금이 있는 대회"는 비상업 조건이 애매해질 수 있어서, 각 라이선스를 아래처럼 확인했습니다. 법률 자문은 아니고, 애매한 부분은 대회 주최 측이나 지도교수님께 확인하세요.

대상	라이선스 (직접 확인)	우리에게 뜻하는 것
RailSafeNet 코드	MIT	복사·수정·상업 이용 모두 허용. 가져온 코드에 저작권 문구 (Copyright (c) 2024 Ondřej Valach)와 MIT 전문을 남겨야 함
Ultralytics YOLO (코드와 학습한 가중치)	AGPL-3.0 (유료 Enterprise 별도)	우리가 학습시킨 모델도 AGPL. 결과물을 배포하거나 서비스로 제공하면 전체 소스를 AGPL로 공개해야 함. Ultralytics는 대학 과제·연구를 AGPL 적합 사례로 들고 있음
ultralyticsplus (RailSafeNet이 YOLO를 부르는 래퍼)	GPL-3.0	쓰지 않고 공식 `ultralytics` 패키지를 바로 쓰면 됨
OpenCV (4.5 이상)	Apache 2.0	제약 거의 없음. 출처 표기만
DepthAI (OAK-D SDK)	MIT	제약 거의 없음
SegFormer 가중치 (RailSafeNet의 레일 분할 모델)	NVIDIA Source Code License	"연구 또는 평가 목적"의 비상업 사용만 허용. RailSafeNet이 공개한 파인튜닝 가중치도 여기에 묶임
RailSem19 데이터	이미지 CC BY-NC 4.0, 라벨 CC BY-NC-SA 4.0 + 추가 조건	비상업·출처 표기 필수. 데이터를 다른 사람에게 직접 나눠주면 안 됨(팀원도 각자 가입 권장), 저장소에 올리지 말 것

우리 팀에 맞는 대응은 네 가지입니다.

- **저장소를 공개하고 AGPL-3.0으로 두기.** 이러면 YOLO 조건이 자연스럽게 충족되고, MIT·Apache 코드를 섞어 써도 문제없습니다.
- **출처를 보고서·발표자료·README에 명시하기.** "RailSafeNet(Valach, 2025)의 궤간 기반 거리 구역 방식을 참고"처럼 무엇을 가져왔고 무엇을 우리가 만들었는지 나눠 적으면, 라이선스뿐 아니라 대회 심사의 표절 시비도 막을 수 있습니다.
- **SegFormer 가중치는 쓰지 않기.** 로드맵대로 YOLO-seg로 레일을 직접 학습하면 비상업 조항을 피하고, 라즈베리파이에서도 더 가볍게 돕니다.

- **대회 요강 확인하기.** 오픈소스 사용 허용 여부, 결과물의 권리(주최 측에 넘기는지)를 보세요. 상금이 있어도 대학 비교과 활동 자체는 보통 비상업·연구 목적에 가깝지만, 주최 측이 결과물을 상업적으로 쓴다면 RailSem19·SegFormer의 비상업 조건에 걸릴 수 있습니다.

3. RailSafeNet은 어떻게 동작하고, 얼마나 가져다 쓸까

RailSafeNet은 "사진 한 장에서 물체가 레일로부터 옆으로 몇 m 떨어져 있나"를 판정하는 연구 코드입니다. 우리 목표의 절반(횡방향 위험 구역)을 잘 보여주지만, 앞쪽 거리·속도·충돌 시간은 다루지 않습니다. 그래서 그대로 돌리는 기반보다는 핵심 아이디어를 가져와 우리 구조에 다시 구현하는 참고용으로 추천합니다.

동작 순서 (`TheDistanceAssessor.py` 기준)

1. **레일 분할**: SegFormer-B3 모델이 사진의 모든 픽셀에 "레일 / 선로 / 기타"를 칠합니다(RailSem19으로 학습).
2. **물체 검출**: YOLOv8s가 사람·자전거·차·트럭·동물(움직이는 물체)과 가방·캐리어 등(정지 물체)을 찾습니다.
3. **선로 가장자리 찾기**: 화면을 가로로 여러 줄(기본 15줄) 잘라, 각 줄에서 왼쪽·오른쪽 레일의 x 좌표를 찾고 튀는 값을 걸러냅니다.
4. **줄마다 자 만들기**: 그 줄에서 두 레일 사이가 N 픽셀이면, 1픽셀 = 1435 mm ÷ N 입니다. 가까운 줄은 1픽셀이 작고 먼 줄은 큽니다.
5. **위험 구역 그리기**: 그 자로 레일 바깥쪽 0.65 m, 1.0 m, 2.0 m 지점을 줄마다 찍고, 이어서 다각형 3개를 만듭니다(끝은 직선으로 연장).
6. **박스 판정**: 물체 박스의 밑부분 점들이 어느 다각형 안에 있는지에 따라 빨강·주황·노랑, 밖이면 초록을 붙입니다.

참고할 점과 바꿔야 할 점

항목	RailSafeNet	우리가 바꿔야 할 것
횡방향 거리(선로 옆)	궤간 1.435 m로 줄마다 픽셀→mm 환산. 매립형 레일에도 통함	**그대로 참고.** 다만 기준을 "레일 바깥"이 아니라 "차체 폭/2 + 여유"로 바꿔 실제 충돌 통로에 맞춤
앞쪽 거리(몇 m 앞인가)	없음	OAK-D 깊이(가까운 곳) + 궤간 픽셀 폭으로 단안 추정(먼 곳) 추가
시간 축	사진 한 장씩 따로 처리. 추적·속도 없음	YOLO `track`으로 ID 유지, 속도·TTC·선로 진입 예측 추가
위험 단계	"어느 구역 안에 있나"만으로 색 결정	구역 × TTC × 필요 감속도로 4단계(4장)

항목	RailSafeNet	우리가 바꿔야 할 것
레일 모델	SegFormer-B3 (무겁고 비상업 조건)	YOLO-seg로 대체, 모형 시험장은 OpenCV로 테이프 선로 검출
실행 환경	PC GPU, 1920×1080·파일 경로가 코드에 박혀 있음	설정 파일로 빼고, 라즈베리파이 5에서 10 FPS 이상인지 측정
코드 형태	900줄짜리 파일 하나, 예외 처리 적음	로드맵 폴더 구조(`rails.py` , `distance.py` , `risk.py`)에 나눠 다시 짜기

한마디로, 3~5번 단계(궤간으로 자 만들고 구역 그리기)가 가장 가치 있는 부분이고, 나머지는 우리가 더 만들어야 합니다. 논문(arXiv 2509.12125)을 보고서의 선행연구 비교 대상으로 쓰기 좋습니다.

4. 내일 논의할 로직 초안

핵심은 "통로 안에 있나(옆 거리)"와 "몇 초 뒤 닿나(앞 거리 ÷ 속도)"를 따로 구해서 겹쳐 보는 것입니다. 아래 숫자는 로드맵의 제안 초깃값이고, 모형 시험으로 조정합니다.

한 프레임의 처리 순서

1. **입력**: OAK-D에서 컬러 영상, 깊이, YOLO 검출 결과를 받는다. 하드웨어가 오기 전에는 OpenCV로 녹화 영상을 읽고 PC에서 YOLO를 돌린다.
2. **선로 검출**: 두 레일을 찾고 가운데 기준선을 만든다. 모형은 OpenCV(색 필터 + HoughLinesP), 실제 영상은 YOLO-seg. 2~3 프레임마다 한 번 하고 사이는 이전 값을 쓴다.
3. **자 만들기**: 화면의 줄(y)마다 레일 사이 픽셀 폭을 재서 "1픽셀 = 몇 m"를 구한다(RailSafeNet 방식).
4. **검출·추적**: 사람·자전거·차량만 남기고, 같은 물체에 같은 ID를 붙인다.
5. **거리**: 박스 밑변 가운데(발 위치)를 기준으로 앞 거리 Z와 옆 거리 d_lat를 구한다.
6. **움직임**: 최근 5~10 프레임의 Z·d_lat 변화로 다가오는 속도와 선로 쪽 옆 속도를 구하고, TTC와 필요 감속도를 계산한다.
7. **단계 판정**: 물체마다 0~3단계를 정하고, 화면·경보는 가장 높은 물체 하나를 따른다.
8. **출력**: 화면에 통로·박스·숫자를 그리고 부저·LED에 단계 숫자만 넘긴다.

거리 계산 (카메라 한 대 기준)

카메라의 초점거리를 픽셀 단위로 구해 둡니다. 1920 px, 수평 화각 60°면 약 1663 px입니다.

f(px) = (영상 가로 픽셀 ÷ 2) ÷ tan(수평 화각 ÷ 2)

물체 발 위치 줄에서 레일 사이가 g 픽셀이면 앞 거리와 옆 거리는 이렇습니다. 예를 들어 100 m 앞에서는 g ≈ 24 px이라 1 px만 틀려도 약 4 m 오차가 납니다.

앞 거리 Z ≈ f(px) × 1.435 ÷ g

옆 거리 d_lat = (물체 x − 기준선 x) × 1.435 ÷ g

가까운 곳(약 8 m 이내)은 OAK-D 깊이값을 우선 쓰고, 레일이 가려지면 사람 키(약 1.7 m)를 같은 식에 넣는 방법을 백업으로 둡니다. 충돌 시간과 필요 감속도는 다음과 같고, v는 다가오는 속도, t_r은 운전자 반응시간(예: 1초)입니다.

TTC = Z ÷ v

필요 감속도 a_req = v² ÷ (2 × (Z − v × t_r))

예: 50 km/h(13.9 m/s)로 달리는데 50 m 앞에 사람이 있으면 TTC는 3.6초지만 필요 감속도가 약 2.7 m/s²로 상용 제동(예시 1.2 m/s²)을 넘습니다. 그래서 TTC만 보면 "경고"지만 실제로는 "위험"입니다.

위험 단계 기준 (제안)

통로는 기준선에서 양쪽으로 "차체 폭/2 + 여유 0.5 m"입니다. 차체 폭 2.65 m를 가정하면 약 ±1.8 m이고, 실제 차량 사양으로 바꿉니다.

단계	조건 (하나라도 만족)	출력
3 위험	통로 안 + (TTC ⩽ 3 s 또는 필요 감속도 ⩾ 1.2 m/s²)	빨간 테두리, 연속음, 영상 기록
2 경고	통로 안 + TTC ⩽ 5 s / 통로 밖이지만 선로로 다가오고, 통로 진입 예상 시간 ⩽ TTC ⩽ 5 s	주황 박스, TTC 강조, 단속음, "감속 권장"
1 주의	통로 안 + TTC > 5 s / 통로 경계에서 1 m 이내(완충 구역)	노란 박스 + 거리
0 안전	나머지	초록 박스
감시 불가	선로 검출 실패 또는 카메라 끊김	회색 표시 (안전 쪽으로 실패)

깜빡임을 막기 위해 단계는 올릴 때는 2 프레임 연속이면 바로, 내릴 때는 1초 동안 낮은 상태가 유지될 때만 내립니다. 거리·속도도 최근 5 프레임 중앙값으로 평활합니다.

```
def risk_level(obj, tram_speed, cfg):
    if obj.in_corridor:
        if obj.ttc <= cfg.ttc_danger or obj.a_req >= cfg.a_service:
            return 3  # 위험
        if obj.ttc <= cfg.ttc_warn:
            return 2  # 경고
        return 1      # 주의
    if obj.approaching and obj.time_to_enter <= obj.ttc <= cfg.ttc_warn:
        return 2      # 곧 들어올 사람
    if obj.d_edge <= cfg.buffer_m:
        return 1
    return 0
```

5. 내일 모임에서 정할 것

- [ ] 저장소를 공개로 두고 AGPL-3.0으로 지정할지 (대회 요강의 오픈소스·결과물 권리 조항 확인 포함)
- [ ] RailSafeNet은 참고만 하고 우리 폴더 구조로 다시 짤지, 코드를 일부 복사할지 (복사 시 MIT 문구 유지)
- [ ] 위험 통로 폭: 차체 폭과 여유값 (초깃값 ±1.8 m, 모형은 ±18 cm)
- [ ] TTC 3초·5초, 상용 감속도 1.2 m/s², 반응시간 1초를 초깃값으로 쓸지
- [ ] 감시 대상 클래스: 사람·자전거·차량 외에 동물·낙하물을 넣을지
- [ ] 이번 주 역할: 공개 트램 영상 확보, PC에서 YOLO `track` 실행, RailSem19 가입 신청

출처

- RailSafeNet 저장소 (코드·LICENSE.txt·requirements 직접 읽음)
- RailSafeNet 논문, arXiv 2509.12125
- Ultralytics 라이선스 안내
- ultralyticsplus LICENSE
- OpenCV LICENSE
- DepthAI LICENSE
- NVIDIA SegFormer LICENSE
- RailSem19 라이선스
- 팀 로드맵(프로젝트 파일 로드맵.pdf): 하드웨어 구성, 4단계 경보, TTC·감속도 초깃값